

Competitive Programming

Section 28: DP I Mixed Contest

Competitive Programming Course

2026-08-05

Table of contents

1	Why a mixed DP contest is different	3
2	Complete local contest set	4
3	Harder second pass	5
4	State diagnosis, not an answer key	6
5	Validate a recurrence	7
6	Complexity discipline	8
7	Implementation habits	9
8	Exercises	10

1 Why a mixed DP contest is different

The hard part is not filling a table. It is deciding whether a state can retain all information needed for the future while discarding everything else.

2 Complete local contest set

Problem	Title
A	Training Schedule
B	Two Carts
C	Prerequisite Tree
D	Circular Exhibition
E	One Diagonal Path
F	Matrix Chain
G	Hamiltonian Route
H	Sliding Jump Cost
I	Grouped Cargo
J	Minimum Starting Energy
K	Last Crystal Removed
L	Weighted Tree Guards

Problems E, F, and G add dedicated grid, interval, and bitmask DP tasks. Treat H through L are a harder second pass over one-dimensional, knapsack, grid, interval, and tree DP. Derive each state from its contract before consulting the editorial.

3 Harder second pass

The added exercises are designed around five questions that do not arise as strongly in A–G:

Problem	Additional question
H. Sliding Jump Cost	Can the repeated transition aggregate be maintained incrementally?
I. Grouped Cargo	Which layer must remain unchanged to prevent taking two choices from one group?
J. Minimum Starting Energy	Is the useful state easier to define backward from the destination?
K. Last Crystal Removed	Does choosing the final action make the interval boundary stable?
L. Weighted Tree Guards	What information must a child communicate about dependence on its parent?

For H, first write the direct $O(nk)$ recurrence. Optimization should preserve that recurrence, not replace its meaning. For I, distinguish an item layer from a group layer. For J, notice that maximum energy upon arrival is not sufficient to describe how much energy a suffix will require. For K, attempting to choose the first removal makes the new neighbors depend on decisions outside each subproblem; reverse the viewpoint. For L, a Boolean “guard or no guard” state cannot distinguish a protected vertex from one still waiting for its parent.

4 State diagnosis, not an answer key

For each local task, answer these questions before writing dp:

```
What decision is made next?  
Which facts about the past can affect later decisions?  
What does one state mean in a complete sentence?  
What is the base case, and why is it true?  
Which earlier states can transition into this state?
```

Do not choose a state because it resembles a familiar template. A good state has a precise meaning and no irrelevant history. The local editorial gives the specific states and transitions after you have tried to derive them.

5 Validate a recurrence

For every transition, prove two directions: every legal construction reaches a state through one listed transition, and every listed transition produces a legal construction. Then decide the iteration order from dependencies: a state must be finalized before later states read it.

Use a tiny brute-force oracle for small inputs. Compare its answer to the DP while changing one feature at a time: an empty choice, a maximum budget, an infeasible prerequisite, or equal values. A mismatch usually exposes a state definition error rather than a syntax error.

6 Complexity discipline

Multiply the number of states by transitions examined per state. If one state dimension follows a large numeric bound, ask whether the bound is genuinely small enough. If the input is a tree or graph, include the total work across all edges and merges; a locally plausible transition may still be quadratic.

7 Implementation habits

Initialize unreachable states explicitly and keep the meaning of the sentinel consistent. Use a wide type for scores and sums. With in-place updates, write down whether reading a value updated in this iteration would illegally reuse the current item or decision.

8 Exercises

Attempt A through G from first principles, then H through L as the harder reinforcement set. For each, submit a one-sentence state definition, a base case, one transition, and the derived complexity before opening the editorial.